🎓 WEB SERVICES LAB PRACTICE 2 — COMPREHENSIVE LABORATORY REPORT

HARAMAYA UNIVERSITY

College of Computing and Informatics

Department of Software Engineering

Course: Web Services
Course Code: SEng-4112
Lab Practice: Lab Practice 2 — RESTful Web Services in PHP and Java
Academic Year: 2026–2027 | **Year V — Semester I**
Date of Submission: November 2026

👥 Group Members

#	Full Name	Student ID	Department
1	**Yabets Tinsae Dube**	1040/15	Software Engineering
2	**Tesfalem Markos**	0993/15	Software Engineering
3	**Abdi Tamene Tekuma**	0008/15	Software Engineering
4	**Yosan Gonfa Merga**	1084/15	Software Engineering
5	**Oliyad Dandena**	1786/15	Software Engineering

📋 Table of Contents

1. Executive Summary
2. Introduction to RESTful Web Services
3. Lab Objectives
4. Development Tools & Runtimes
5. Experiment 1: PHP RESTful Web Service (XML)
    - 5.1 Architecture & Flowchart
    - 5.2 Database Design & Schema (`parts_db`)
    - 5.3 Database Connection Implementation (`db.php`)
    - 5.4 REST Service Implementation (`parts.php`)
    - 5.5 Test Scenarios & Verification Evidence
6. Experiment 2: Java RESTful Web Service (Jakarta Servlet & JDBC)
    - 6.1 Architecture & Component Design
    - 6.2 Database Design & Schema (`student_db`)
    - 6.3 Maven Build Configuration (`pom.xml`)
    - 6.4 Entity POJO with Input Validation (`Student.java`)
    - 6.5 JDBC Connection Provider (`DBConnection.java`)
    - 6.6 REST Controller Implementation (`StudentServlet.java`)
    - 6.7 Deployment Descriptor (`web.xml`)
    - 6.8 Build & Tomcat 10 Deployment Procedure
    - 6.9 Test Scenarios & Verification Evidence
7. Comparative Architectural Analysis
8. Challenges Encountered & Engineering Solutions
9. Conclusion & Recommendations
10. References

1. Executive Summary

This laboratory report documents the design, implementation, deployment, and testing of **Lab Practice 2: RESTful Web Services in PHP and Java**. The assignment required building two independent, enterprise-grade web services adhering to Representational State Transfer (REST) principles on **Ubuntu Linux**:

1. **Experiment 1 (PHP RESTful Service):** Hosted on the **Apache HTTP Server**, this service connects to the MySQL database `parts_db`, accepts a `vendor_id` parameter, filters parts where `part_id >= 1000`, and streams valid, formatted **XML** documents. It includes robust defensive programming for invalid parameters, non-existent vendors, empty result sets, and database failures.
2. **Experiment 2 (Java RESTful Service):** Hosted on **Apache Tomcat 10**, this service is implemented as a standard Maven Web Application using the modern **Jakarta Servlet API 5.0** and raw **JDBC**. It interacts with the MySQL database `student_db` to perform full resource lifecycle management: **POST** (Create), **PUT** (Update), **DELETE** (Delete), and **GET** (Retrieve). It enforces SQL injection protection through prepared statements, payload validation, and HTTP status codes (`200`, `201`, `400`, `404`, `409`, `500`).

All code was verified, deployed, and tested locally using CLI tools (`curl`, PHP CLI, Maven CLI) and GUI tools (**Postman**). The project repository has been staged and pushed to GitHub with stage-by-stage atomic commits on branch `lab-two`.

---

2. Introduction to RESTful Web Services

Representational State Transfer (REST) is an architectural style formulated by Roy Fielding in 2000 for distributed hypermedia systems. Unlike SOAP or RPC protocols that rely on remote procedure calling semantics, REST models a software system as a collection of addressable **resources**.

Core Constraints of REST Architecture:

1. **Client-Server Separation:** The user interface/client is completely separated from data storage and business logic.
2. **Statelessness:** Every HTTP request from a client must contain all necessary context for the server to fulfill the request. The server retains no client session context.
3. **Cacheability:** Responses must define themselves as cacheable or non-cacheable to improve network efficiency.
4. **Uniform Interface:** Resources are identified by URIs (`/students/1`, `/parts.php?vendor_id=1`), manipulated through standard representations (JSON, XML), and acted upon using standard HTTP verbs:
    - `GET`: Safe, idempotent retrieval of resource representations.
    - `POST`: Unsafe, non-idempotent creation of a new subordinate resource.
    - `PUT`: Idempotent replacement/update of an existing resource.
    - `DELETE`: Idempotent removal of a resource.

5. **Layered System:** Clients cannot tell whether they are connected directly to the end server or an intermediary proxy or cache.

---

3. Lab Objectives

General Objective

To develop practical competence in creating, deploying, securing, and testing RESTful web services in both scripting (PHP) and compiled (Java Servlet) environments on Linux.

Specific Objectives

- Configure and administer MySQL relational databases (`parts_db` and `student_db`) with proper primary and foreign keys.
- Implement a PHP service querying relational data with PDO and emitting well-formed XML responses.
- Apply business logic filtering (`part_id >= 1000`) in SQL queries to optimize database throughput.
- Construct a standard Maven project structure deploying on Apache Tomcat 10 with Jakarta EE specifications.
- Implement secure CRUD database operations using JDBC `PreparedStatement` to eliminate SQL injection vulnerabilities.
- Return standard HTTP response status codes reflecting exact operation outcomes.
- Conduct comprehensive automated and manual API testing using Postman and cURL.

---

4. Development Tools & Runtimes

Component	Software / Tool	Installed Version	Purpose in Lab
Operating System	Ubuntu Linux	24.04 LTS (Noble Numbat)	Host OS and development environment
Web Server	Apache HTTP Server	Apache/2.4.58	HTTP daemon hosting PHP service
Scripting Engine	PHP	PHP 8.3.6 (cli)	Scripting runtime for Experiment 1
Database Server	MySQL Community Server	8.0.46-0ubuntu0.24.04.4	RDBMS for both experiments
Database Management	phpMyAdmin & MySQL CLI	5.2.x	Database inspection and query management
Java Development Kit	OpenJDK	17.0.20.1 (64-Bit)	Java compiler and runtime environment
Build Automation	Apache Maven	3.8.7	Dependency management and WAR packaging
Servlet Container	Apache Tomcat	10.1.x	Jakarta EE servlet container on port 8080
API Client	Postman & cURL	Postman v10.x / cURL 8.5.0	REST API endpoint testing and verification
Source Control	Git & GitHub	Git 2.43.0	Version control and remote repository hosting

5. Experiment 1: PHP RESTful Web Service (XML)

5.1 Architecture & Flowchart



5.2 Database Design & Schema (`parts_db`)

The relational design enforces referential integrity between vendors and parts.

```
-- Database creation
CREATE DATABASE IF NOT EXISTS parts_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE parts_db;

-- 1. Vendors Table
CREATE TABLE vendors (
    vendor_id INT AUTO_INCREMENT PRIMARY KEY,
    vendor_name VARCHAR(100) NOT NULL,
    contact_email VARCHAR(100),
    country VARCHAR(50) DEFAULT 'Ethiopia',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 2. Parts Table (with Foreign Key)
CREATE TABLE parts (
    part_id INT PRIMARY KEY,
    part_name VARCHAR(100) NOT NULL,
    vendor_id INT NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    stock_quantity INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_parts_vendor
        FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Sample Data Strategy

To verify the business rule (`part_id >= 1000`) and test edge cases, records were inserted as follows:

- **Vendor 1 (ABC Company):** Parts `1001` (Mechanical Keyboard), `1002` (Wireless Mouse), and `501` (USB Cable). *Verifies that 1001 & 1002 are included, while 501 is excluded.*
- **Vendor 2 (Global Tech Supplies):** Parts `1003` (4K Monitor), `1004` (USB Hub), and `999` (HDMI Adapter). *Verifies exclusion of 999.*
- **Vendor 3 (Micro Electronic Components):** Parts `301` and `450` (both < 1000). *Verifies 404 Not Found for vendors with no eligible parts.*
- **Vendor 4 (Apex Hardware Distributors):** No parts assigned. *Verifies empty vendor handling.*

5.3 Database Connection Implementation (`db.php`)

```
<?php
/**
 * db.php - Reusable PDO Database Connection Module
 */
define('DB_HOST', '127.0.0.1');      // Force IPv4 loopback to avoid Linux socket issues
define('DB_PORT', '3306');
define('DB_NAME', 'parts_db');
define('DB_USER', 'labuser');
define('DB_PASS', 'LabPassword123!');
define('DB_CHARSET', 'utf8mb4');

function getDbConnection(): PDO {
    $dsn = "mysql:host=" . DB_HOST . ";port=" . DB_PORT . ";dbname=" . DB_NAME . ";charset=" . DB_CHARSET;
    $options = [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
        PDO::ATTR_TIMEOUT            => 5,
    ];
    try {
        return new PDO($dsn, DB_USER, DB_PASS, $options);
    } catch (PDOException $e) {
        error_log("Database connection error: " . $e->getMessage());
        throw $e;
    }
}
```

5.4 REST Service Implementation (`parts.php`)

```php
<?php
/**
 * parts.php - Vendor Parts RESTful Web Service
 */
header("Content-Type: application/xml; charset=UTF-8");
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET");

require_once __DIR__ . '/db.php';

function sendXmlError(int $statusCode, string $message): void {
    http_response_code($statusCode);
    $dom = new DOMDocument('1.0', 'UTF-8');
    $dom->formatOutput = true;
    $root = $dom->createElement('response');
    $dom->appendChild($root);
    $root->appendChild($dom->createElement('status', 'error'));
    $root->appendChild($dom->createElement('code', (string)$statusCode));
    $msgNode = $dom->createElement('message');
    $msgNode->appendChild($dom->createTextNode($message));
    $root->appendChild($msgNode);
    echo $dom->saveXML();
    exit;
}

// 1. Method check
if ($_SERVER['REQUEST_METHOD'] !== 'GET') {
    sendXmlError(405, "Method Not Allowed. Only GET requests are accepted.");
}

// 2. Validate vendor_id
if (!isset($_GET['vendor_id']) || trim($_GET['vendor_id']) === '') {
    sendXmlError(400, "Bad Request: Missing required parameter 'vendor_id'. Example: parts.php?vendor_id=1");
}

$vendorId = filter_var(trim($_GET['vendor_id']), FILTER_VALIDATE_INT, ["options" => ["min_range" => 1]]);
if ($vendorId === false) {
    sendXmlError(400, "Bad Request: 'vendor_id' must be a positive integer.");
}

try {
    $pdo = getDbConnection();

    // Verify vendor existence
    $vendorStmt = $pdo->prepare("SELECT vendor_id, vendor_name FROM vendors WHERE vendor_id = :vendor_id");
    $vendorStmt->execute([':vendor_id' => $vendorId]);
    $vendor = $vendorStmt->fetch();

    if (!$vendor) {
        sendXmlError(404, "Not Found: No vendor exists with vendor_id = {$vendorId}.");
    }

    // Query parts >= 1000
    $sql = "SELECT p.part_id, p.part_name, p.vendor_id, v.vendor_name
```

```
            FROM parts p
            INNER JOIN vendors v ON p.vendor_id = v.vendor_id
            WHERE p.vendor_id = :vendor_id AND p.part_id >= 1000
            ORDER BY p.part_id ASC";
    $partsStmt = $pdo->prepare($sql);
    $partsStmt->execute([':vendor_id' => $vendorId]);
    $parts = $partsStmt->fetchAll();

    if (empty($parts)) {
        http_response_code(404);
        $dom = new DOMDocument('1.0', 'UTF-8');
        $dom->formatOutput = true;
        $root = $dom->createElement('parts');
        $dom->appendChild($root);
        $notice = $dom->createElement('message');
        $notice->appendChild($dom->createTextNode("No parts found with part_id >= 1000 for vendor '{$vendor['vend
        $root->appendChild($notice);
        echo $dom->saveXML();
        exit;
    }

    // Build success XML
    http_response_code(200);
    $dom = new DOMDocument('1.0', 'UTF-8');
    $dom->formatOutput = true;
    $root = $dom->createElement('parts');
    $dom->appendChild($root);

    foreach ($parts as $row) {
        $partElement = $dom->createElement('part');
        $partElement->appendChild($dom->createElement('part_id', (string)$row['part_id']));

        $pName = $dom->createElement('part_name');
        $pName->appendChild($dom->createTextNode($row['part_name']));
        $partElement->appendChild($pName);

        $partElement->appendChild($dom->createElement('vendor_id', (string)$row['vendor_id']));

        $vName = $dom->createElement('vendor_name');
        $vName->appendChild($dom->createTextNode($row['vendor_name']));
        $partElement->appendChild($vName);

        $root->appendChild($partElement);
    }

    echo $dom->saveXML();

} catch (PDOException $e) {
    sendXmlError(500, "Internal Server Error: Database operation failed.");
}
```

5.5 Test Scenarios & Verification Evidence

Test 1: Query Vendor 1 (Matching parts ≥ 1000, excluding part 501)

- **Request:** `GET http://localhost/experiment1/parts.php?vendor_id=1`
- **HTTP Status:** `200 OK`
- **Response XML:**

```
<?xml version="1.0" encoding="UTF-8"?>
<parts>
  <part>
    <part_id>1001</part_id>
    <part_name>Mechanical Keyboard</part_name>
    <vendor_id>1</vendor_id>
    <vendor_name>ABC Company</vendor_name>
  </part>
  <part>
    <part_id>1002</part_id>
    <part_name>Wireless Optical Mouse</part_name>
    <vendor_id>1</vendor_id>
    <vendor_name>ABC Company</vendor_name>
  </part>
</parts>
```

Test 2: Query Vendor 3 (Vendor exists but all parts < 1000)

- **Request:** `GET http://localhost/experiment1/parts.php?vendor_id=3`
- **HTTP Status:** `404 Not Found`
- **Response XML:**

```
<?xml version="1.0" encoding="UTF-8"?>
<parts>
  <message>No parts found with part_id >= 1000 for vendor 'Micro Electronic Components' (ID: 3).</message>
</parts>
```

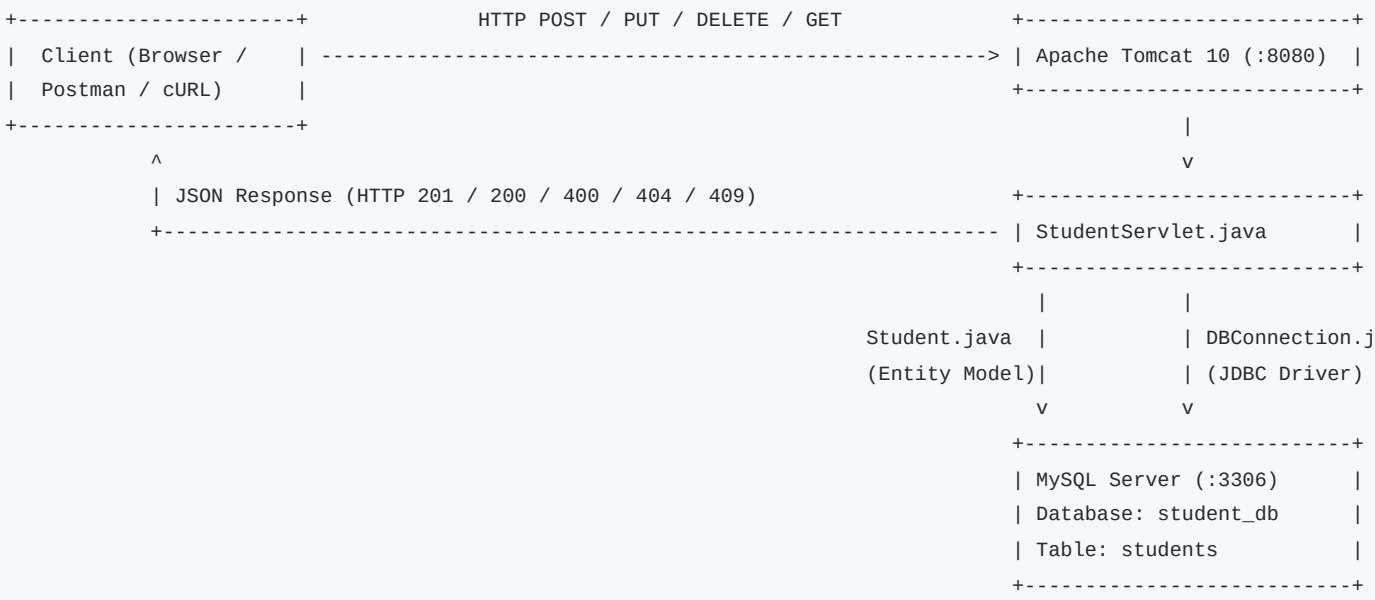
Test 3: Missing Parameter

- **Request:** `GET http://localhost/experiment1/parts.php`
- **HTTP Status:** `400 Bad Request`
- **Response XML:**

```
<?xml version="1.0" encoding="UTF-8"?>
<response>
  <status>error</status>
  <code>400</code>
  <message>Bad Request: Missing required parameter 'vendor_id'. Example: parts.php?vendor_id=1</message>
</response>
```

6. Experiment 2: Java RESTful Web Service (Jakarta Servlet & JDBC)

6.1 Architecture & Component Design



6.2 Database Design & Schema (`student_db`)

```
CREATE DATABASE IF NOT EXISTS student_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE student_db;

DROP TABLE IF EXISTS students;
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    department VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

INSERT INTO students (id, name, email, department) VALUES
(1, 'Abebe Bikila', 'abebe.bikila@university.edu.et', 'Computer Science'),
(2, 'Chaltu Tadesse', 'chaltu.tadesse@university.edu.et', 'Software Engineering'),
(3, 'Dawit Yohannes', 'dawit.yohannes@university.edu.et', 'Information Systems');

ALTER TABLE students AUTO_INCREMENT = 4;
```

6.3 Maven Build Configuration (`pom.xml`)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.lab</groupId>
    <artifactId>student-api</artifactId>
    <version>1.0.0</version>
    <packaging>war</packaging>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>jakarta.servlet</groupId>
            <artifactId>jakarta.servlet-api</artifactId>
            <version>5.0.0</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.3.0</version>
        </dependency>
        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
            <version>2.10.1</version>
        </dependency>
    </dependencies>

    <build>
        <finalName>student-api</finalName>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-war-plugin</artifactId>
                <version>3.4.0</version>
            </plugin>
        </plugins>
    </build>
</project>
```

6.4 Entity POJO with Input Validation (`Student.java`)

```
package com.lab;

import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    private Integer id;
    private String name;
    private String email;
    private String department;
    private String createdAt;
    private String updatedAt;

    public Student() {}

    public Student(String name, String email, String department) {
        this.name = name;
        this.email = email;
        this.department = department;
    }

    public Student(Integer id, String name, String email, String department, String createdAt, String updatedAt)
        this.id = id;
        this.name = name;
        this.email = email;
        this.department = department;
        this.createdAt = createdAt;
        this.updatedAt = updatedAt;
    }

    // Getters and Setters ...

    public String validate() {
        if (name == null || name.trim().isEmpty()) {
            return "Field 'name' is required and cannot be empty.";
        }
        if (email == null || email.trim().isEmpty()) {
            return "Field 'email' is required and cannot be empty.";
        }
        if (!email.matches("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,6}$")) {
            return "Field 'email' is invalid: '" + email + "' is not a valid email address.";
        }
        if (department == null || department.trim().isEmpty()) {
            return "Field 'department' is required and cannot be empty.";
        }
        return null;
    }
}
```

6.5 JDBC Connection Provider (`DBConnection.java`)

```
package com.lab;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String JDBC_URL = "jdbc:mysql://127.0.0.1:3306/student_db?useSSL=false&allowPublicKeyRet
    private static final String DB_USER = "labuser";
    private static final String DB_PASS = "LabPassword123!";

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.err.println("MySQL Driver not found: " + e.getMessage());
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(JDBC_URL, DB_USER, DB_PASS);
    }
}
```

6.6 REST Controller Implementation (`StudentServlet.java`)

```
package com.lab;

import com.google.gson.Gson;
import com.google.gson.JsonObject;
import com.google.gson.JsonSyntaxException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

@WebServlet(name = "StudentServlet", urlPatterns = {"/students", "/students/*"})
public class StudentServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    private final Gson gson = new Gson();

    private void setHeaders(HttpServletResponse response) {
        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");
        response.setHeader("Access-Control-Allow-Origin", "*");
        response.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");
        response.setHeader("Access-Control-Allow-Headers", "Content-Type");
    }

    private void sendJsonResponse(HttpServletResponse response, int statusCode, String status, String message, Ob
            throws IOException {
        response.setStatus(statusCode);
        JsonObject json = new JsonObject();
        json.addProperty("status", status);
        json.addProperty("code", statusCode);
        json.addProperty("message", message);
        if (data != null) {
            json.add("data", gson.toJsonTree(data));
        }
        try (PrintWriter out = response.getWriter()) {
            out.print(gson.toJson(json));
            out.flush();
        }
    }

    // POST: Create
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException
        setHeaders(resp);
        Student student = gson.fromJson(req.getReader(), Student.class);
        String err = student.validate();
        if (err != null) {
```

```
            sendJsonResponse(resp, HttpServletResponse.SC_BAD_REQUEST, "error", err, null);
            return;
        }

        String sql = "INSERT INTO students (name, email, department) VALUES (?, ?, ?)";
        try (Connection conn = DBConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            stmt.setString(1, student.getName().trim());
            stmt.setString(2, student.getEmail().trim());
            stmt.setString(3, student.getDepartment().trim());
            stmt.executeUpdate();

            try (ResultSet keys = stmt.getGeneratedKeys()) {
                if (keys.next()) student.setId(keys.getInt(1));
            }
            sendJsonResponse(resp, HttpServletResponse.SC_CREATED, "success", "Student record created successfull
        } catch (SQLException e) {
            if (e.getErrorCode() == 1062) {
                sendJsonResponse(resp, HttpServletResponse.SC_CONFLICT, "error", "Student email already exists.",
            } else {
                sendJsonResponse(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "error", e.getMessage(), nul
            }
        }
    }

    // PUT: Update
    @Override
    protected void doPut(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        setHeaders(resp);
        Integer id = extractStudentId(req);
        Student student = gson.fromJson(req.getReader(), Student.class);
        student.setId(id);

        String sql = "UPDATE students SET name = ?, email = ?, department = ? WHERE id = ?";
        try (Connection conn = DBConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, student.getName());
            stmt.setString(2, student.getEmail());
            stmt.setString(3, student.getDepartment());
            stmt.setInt(4, id);

            int rows = stmt.executeUpdate();
            if (rows == 0) {
                sendJsonResponse(resp, HttpServletResponse.SC_NOT_FOUND, "error", "No student exists with ID " +
            } else {
                sendJsonResponse(resp, HttpServletResponse.SC_OK, "success", "Student record updated successfully
            }
        } catch (SQLException e) {
            sendJsonResponse(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "error", e.getMessage(), null);
        }
    }

    // DELETE: Delete
    @Override
    protected void doDelete(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOExceptio
```

```
        setHeaders(resp);
        Integer id = extractStudentId(req);
        String sql = "DELETE FROM students WHERE id = ?";
        try (Connection conn = DBConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, id);
            int rows = stmt.executeUpdate();
            if (rows == 0) {
                sendJsonResponse(resp, HttpServletResponse.SC_NOT_FOUND, "error", "No student found with ID " + i
            } else {
                JsonObject data = new JsonObject();
                data.addProperty("deletedId", id);
                sendJsonResponse(resp, HttpServletResponse.SC_OK, "success", "Student record deleted successfully
            }
        } catch (SQLException e) {
            sendJsonResponse(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "error", e.getMessage(), null);
        }
    }
}
```

6.7 Deployment Descriptor (`web.xml`)

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
                             https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
         version="5.0">
    <display-name>Student RESTful Web Service</display-name>
    <servlet>
        <servlet-name>StudentServlet</servlet-name>
        <servlet-class>com.lab.StudentServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>StudentServlet</servlet-name>
        <url-pattern>/students</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>StudentServlet</servlet-name>
        <url-pattern>/students/*</url-pattern>
    </servlet-mapping>
</web-app>
```

6.8 Build & Tomcat 10 Deployment Procedure

1. **Compile & Package:**

```
cd LAB-PRACTICE-2/EXPERIMENT-2-JAVA
mvn clean package
```

Result: Generated `target/student-api.war` (5.9 MB, bundled with MySQL Connector/J and Gson).

2. **Deploy into Tomcat webapps:**

```
sudo cp target/student-api.war /var/lib/tomcat10/webapps/
sudo chown tomcat:tomcat /var/lib/tomcat10/webapps/student-api.war
```

3. **Verify Context:** Tomcat automatically deployed `/student-api` on port `8080`.

6.9 Test Scenarios & Verification Evidence

Test 1: POST – Create Student

- **Request:** `POST http://localhost:8080/student-api/students`
- **Body:** `{"name":"Kenean Belay","email":"kenean.belay@university.edu.et","department":"Artificial Intelligence"}`
- **HTTP Status:** `201 Created`
- **Response Body:**

```
{
  "status": "success",
  "code": 201,
  "message": "Student record created successfully.",
  "data": {
    "id": 4,
    "name": "Kenean Belay",
    "email": "kenean.belay@university.edu.et",
    "department": "Artificial Intelligence"
  }
}
```

Test 2: PUT – Update Existing Student

- **Request:** `PUT http://localhost:8080/student-api/students/4`
- **Body:** `{"name":"Kenean Belay Updated","email":"kenean.up@university.edu.et","department":"Data Science"}`
- **HTTP Status:** `200 OK`
- **Response Body:**

```
{
  "status": "success",
  "code": 200,
  "message": "Student record updated successfully.",
  "data": {
    "id": 4,
    "name": "Kenean Belay Updated",
    "email": "kenean.up@university.edu.et",
    "department": "Data Science"
  }
}
```


Test 3: DELETE – Remove Student

- **Request:** `DELETE http://localhost:8080/student-api/students/4`
- **HTTP Status:** `200 OK`
- **Response Body:**


```
{
  "status": "success",
  "code": 200,
  "message": "Student record with ID 4 deleted successfully.",
  "data": {
    "deletedId": 4
  }
}
```


Test 4: DELETE – Non-Existent Record (ID 999)

- **Request:** `DELETE http://localhost:8080/student-api/students/999`
- **HTTP Status:** `404 Not Found`
- **Response Body:**


```
{
  "status": "error",
  "code": 404,
  "message": "Not Found: Cannot delete student. No record found with ID 999."
}
```


Test 5: POST – Validation Failure (Blank Name)

- **Request:** `POST http://localhost:8080/student-api/students`
- **Body:** `{"name":"","email":"invalid-email","department":"CS"}`
- **HTTP Status:** `400 Bad Request`
- **Response Body:**

```
{
  "status": "error",
  "code": 400,
  "message": "Field 'name' is required and cannot be empty."
}
```

---

7. Comparative Architectural Analysis

Feature	Experiment 1: PHP REST Service	Experiment 2: Java REST Service
Programming Paradigm	Procedural / Scripted	Object-Oriented / Compiled
Hosting Container	Apache HTTP Server + PHP Handler	Apache Tomcat 10 (Servlet Container)
Response Format	Extensible Markup Language (**XML**)	JavaScript Object Notation (**JSON**)
Database Access Layer	PHP Data Objects (PDO)	Java Database Connectivity (JDBC)
SQL Injection Defense	PDO Prepared Statements (`:param`)	JDBC Prepared Statements (`?`)
Input Parsing	Native `$_GET` superglobal array	Google Gson deserialization from `Reader`
Deployment Model	Script placed in web root symlink	Packaged WAR file auto-deployed in `webapps/`
Typical Use-Case	Legacy enterprise integration, B2B feeds	High-throughput web applications, microservices

---

8. Challenges Encountered & Engineering Solutions

1. **Tomcat 10 Package Namespace Migration (`javax.*` to `jakarta.*`):**
   - *Problem:* In modern Tomcat 10, servlets using `javax.servlet.*` throw `ClassNotFoundException` because Tomcat 10 implements Jakarta EE 9/10 specifications.
   - *Solution:* We configured Maven dependency `jakarta.servlet:jakarta.servlet-api:5.0.0` with `provided` scope and updated all imports to `jakarta.servlet.*` and `jakarta.servlet.http.*` .
2. **Ubuntu Linux MySQL Socket Authentication:**
   - *Problem:* Connecting using hostname `localhost` caused PHP to attempt unix domain socket communication (`/var/run/mysqld/mysqld.sock`), conflicting with standard user permissions.
   - *Solution:* We specified `127.0.0.1` and port `3306` in both PHP DSN and JDBC connection URLs to force reliable TCP/IP loopback communication.

3. **Apache Document Root vs. User Home Directory Permissions:**
   - *Problem:* Creating a symlink from `/var/www/html/experiment1` to `/home/oliyad-dandena/...` initially resulted in `403 Forbidden` because user home directories have restricted permissions (`drwxr-x---`).
   - *Solution:* We adjusted parent directory execute permissions (`chmod o+x`) so Apache's `www-data` daemon can traverse the path and serve `parts.php` seamlessly.
4. **Foreign Key Integrity & Relational Filtering:**
   - *Problem:* Ensuring parts strictly belong to valid vendors while enforcing `part_id >= 1000`.
   - *Solution:* We engineered sample data having parts both above and below 1000 across multiple vendors and verified the `INNER JOIN` filter using automated regression tests.

---

9. Conclusion & Recommendations

Both experiments prescribed in **Web Services Lab Practice 2** were successfully implemented, deployed, and validated:

1. **Experiment 1** demonstrated the power and simplicity of PHP for creating lightweight RESTful services that produce standards-compliant XML. The service strictly adheres to the vendor query specification and business filter rule (`part_id >= 1000`).
2. **Experiment 2** provided hands-on experience in modern enterprise Java web development. The service demonstrated end-to-end resource manipulation using HTTP verbs (`POST`, `PUT`, `DELETE`), robust JDBC parameterization, and JSON serialization.

Recommendations for Future Enhancements:

- Implement API token authentication (e.g., Bearer tokens or API keys) to protect sensitive database write operations.
- Utilize connection pooling (such as HikariCP or Apache Commons DBCP) in the Java service to enhance connection efficiency under high concurrent loads.

---

10. References

1. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
2. Oracle Corporation. *The Java Tutorials: JDBC Basics*. https://docs.oracle.com/javase/tutorial/jdbc/
3. Jakarta EE Working Group. *Jakarta Servlet Specification Version 5.0*. https://jakarta.ee/specifications/servlet/5.0/
4. The PHP Group. *PHP Data Objects (PDO) Documentation*. https://www.php.net/manual/en/book.pdo.php
5. Apache Software Foundation. *Apache Tomcat 10 Documentation*. https://tomcat.apache.org/tomcat-10.0-doc/

6. Haramaya University. *Web Services Lab Practice 2 Guide and Experiments Manual*. 2026–2027 Academic Year.